

LABORATORY

LAB 06

Potentiometer LED Dimmer

Combining Analog Input and PWM Output with EduFramework

S32K144 · MaaZEDU

EduFramework

FPT University

Contents

1	Introduction	2
1.1	Lab Overview	2
1.2	Objectives	2
2	Background	2
2.1	From Analog Input to PWM Output	2
2.2	Value Range Mapping	2
3	Hardware Setup	3
3.1	Hardware Used	3
3.2	Pin Mapping	3
3.3	Potentiometer Connection	4
4	EduFramework API	4
5	Lab Exercise	4
5.1	Requirements	4
5.2	Program	5
5.3	Verification	5
6	Extension	5
7	References	6

1 Introduction

1.1 Lab Overview

Lab 04 used the ADC to read an analog signal, while Lab 05 used PWM to control LED brightness. This lab combines these two concepts into a complete processing flow: reading the potentiometer position, converting the ADC value to the PWM range, and using the result to directly control LED brightness.

The potentiometer produces a variable voltage at its middle pin as the knob position changes. This voltage is applied to `ADC0_SE12`, after which `analogRead()` returns a 12-bit ADC value. The result is mapped from the range `0` to `4095` into the range `0` to `255` before being passed to `analogWrite()`.

This lab does not introduce a new analog API. The focus is on combining Analog Input, data processing, and PWM Output into a simple interactive application.

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Describe the processing flow from an analog input signal to a PWM output.
2. Perform linear mapping of a 12-bit ADC value from `0..4095` to the PWM range `0..255`.
3. Combine `analogRead()` and `analogWrite()` in the same application.
4. Build and verify an application that controls LED brightness using a potentiometer.

2 Background

2.1 From Analog Input to PWM Output

An ADC converts an analog voltage into a digital value that can be processed by the program. With 12-bit resolution, there are 4096 different digital codes and the result range can be represented from `0` to `4095` [1]. On the output side, the lab uses `analogWrite()` with a control range from `0` to `255`. Because the ADC value range and the PWM range are different, the ADC result cannot be passed directly to the PWM output and must first be converted to the appropriate range.

The data flow of the lab can be represented as follows:

Potentiometer → `ADC0_SE12` → `analogRead()` → Scaling → `analogWrite()` → `LED_RED`

This organization separates the application into three clear stages: data acquisition, data processing, and output control.

2.2 Value Range Mapping

The lab exercise must convert an ADC value in the range `0..4095` into a PWM value in the range `0..255`. When both ranges start at `0`, the linear mapping can be written as:

$$\text{PWM} = \text{ADC} \times \left(\frac{255}{4095} \right)$$

In the C program, the calculation is performed as:

```
pwmValue = (adcValue * 255) / 4095;
```

Mapping an analog input value into the PWM output range is also used in Arduino's Analog In, Out Serial example, where the Analog Input result is mapped to the 0..255 range before controlling PWM [3].

ADC Value	PWM Value	Control Level
0	0	LED off.
1024	63	Low PWM level.
2048	127	Approximately the middle of the control range.
3072	191	High PWM level.
4095	255	Maximum brightness.

Table 6.2.1. Example mapping from a 12-bit ADC value to the PWM range 0 to 255

As the ADC value increases, the mapped PWM value also increases. Therefore, with the connection used in this lab, moving the potentiometer wiper closer to 3V3 increases the ADC value and makes the LED brighter; moving it closer to GND decreases the ADC value and makes the LED dimmer.

Note

Mapping only converts between two numeric ranges. ADC conversion and PWM principles were already presented in Lab 04 and Lab 05 and are therefore not repeated in this lab.

3 Hardware Setup

3.1 Hardware Used

This lab uses a potentiometer as the Analog Input and the onboard red LED on the MaaZEDU Development Board as the PWM Output.

Hardware	Description
MaaZEDU Development Board	Development board using the S32K144 microcontroller.
Potentiometer	Produces a variable voltage according to the knob position.
Breadboard	Used to arrange the potentiometer and external connections.
Jumper wires	Connect 3V3, GND, and the ADC signal between the potentiometer and the board.
USB Cable	Connects the board to the computer for power and program uploading.

Table 6.3.1. Hardware used in the lab exercise

3.2 Pin Mapping

The lab uses ADC0_SE12 as the Analog Input and LED_RED as the PWM Output. On the S32K144, ADC0 Channel 12 is mapped to PTC14; NXP also uses Channel 12 as the potentiometer input in its S32K144 ADC example [2].

Component	Logical Pin	MCU Pin	Function
Potentiometer middle pin	ADC0_SE12	PTC14	Analog Input / ADC0 Channel 12
Onboard red LED	LED_RED	PTD15	PWM Output

Table 6.3.2. Pin mapping used in the lab exercise

3.3 Potentiometer Connection

The two outer pins of the potentiometer are connected to `3V3` and `GND`. The middle pin (wiper) is connected to `ADC0_SE12` (`PTC14`). When the potentiometer is rotated, the voltage at the middle pin changes between the two supply levels and provides an analog signal to the ADC.

`LED_RED` is an onboard LED, so no external LED or resistor is required.

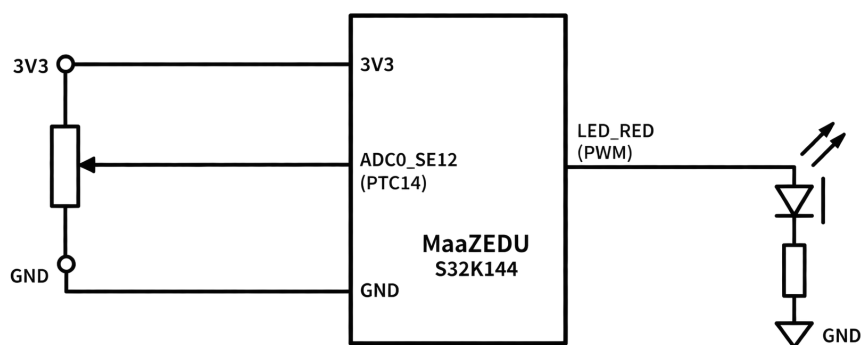


Figure 6.3.1. Potentiometer connection to the Analog Input in Lab 06

4 EduFramework API

Lab 06 does not introduce a new API. The program combines two APIs used in previous labs: `analogRead()` to obtain the ADC value and `analogWrite()` to set the PWM level [4].

The API flow in the application is:

`analogRead(ADC0_SE12) → Scaling 0..4095 → 0..255 → analogWrite(LED_RED, pwmValue)`
`pinMode()` continues to be used to configure `LED_RED` as `OUTPUT`, as in the previous labs. The syntax and parameters of these APIs are not repeated in this lab.

5 Lab Exercise

5.1 Requirements

Build a program that uses a potentiometer to directly control the brightness of `LED_RED`.

The program must satisfy the following requirements:

- Read the potentiometer value through `ADC0_SE12`.
- Map the ADC value from the `0..4095` range to the PWM `0..255` range.
- Use the mapped value to control `LED_RED`.
- Update continuously so that the LED responds to the current potentiometer position.

5.2 Program

In `src/main.c`, implement the program that has been verified on the hardware as follows:

```
#include "Arduino.h"

int main(void)
{
    int adcValue = 0;
    int pwmValue = 0;

    setup();

    pinMode(LED_RED, OUTPUT);

    while (1)
    {
        adcValue = analogRead(ADC0_SE12);

        pwmValue = (adcValue * 255) / 4095;

        analogWrite(LED_RED, pwmValue);
    }

    return 0;
}
```

Listing 6.5.1. Program for controlling LED brightness using a potentiometer

`setup()` initializes the fundamental components of EduFramework, and `pinMode(LED_RED, OUTPUT)` configures the red LED as an output.

In the main loop, `analogRead(ADC0_SE12)` reads the voltage at the middle pin of the potentiometer and returns the ADC value. The following statement:

```
pwmValue = (adcValue * 255) / 4095;
```

performs the mapping from the 12-bit ADC range to the PWM range. The `pwmValue` is then passed directly to `analogWrite()`, so each change in the potentiometer is reflected as a corresponding change in LED brightness.

5.3 Verification

Build and upload the program to the MaaZEDU Development Board. Rotate the potentiometer from a position near `GND` toward `3V3`, then rotate it in the opposite direction and observe `LED_RED`.

EXPECTED RESULT

The brightness of `LED_RED` changes according to the potentiometer position. As the voltage at the middle pin increases, the ADC value and PWM value increase, making the LED brighter. As the voltage at the middle pin decreases, the LED becomes dimmer and can return to the off state at the lower end of the control range.

6 Extension

The main exercise prioritizes direct response between the potentiometer and the LED, so it does not display data inside the loop. To clearly observe the mapping process, the Serial Monitor introduced in Lab 03 can be

used to display both the input ADC value and the processed PWM value. Displaying both the input and output values is also used in Arduino's Analog In, Out Serial example [3] .

The data can be presented in the following format:

```
ADC: 512 | PWM: 31
ADC: 2048 | PWM: 127
ADC: 3584 | PWM: 223
```

The values above only illustrate the display format; the actual values depend on the potentiometer position and the ADC signal at the time of measurement.

Note

If data is sent to the Serial Monitor in every loop iteration, the display rate may be too fast to observe. When implementing the extension, an appropriate update interval can be selected for monitoring purposes.

Extension: Add the Serial Monitor at a baud rate of 9600 and display `adcValue` and `pwmValue` on the same line. Rotate the potentiometer through several positions and verify that the PWM value changes from near 0 to near 255 according to the change in the ADC result.

7 References

- [1] The University of Texas at Austin, *Chapter 7: ADC, Data Acquisition, and Control* , Embedded Systems.
- [2] NXP Semiconductors, *S32K1xx Series Cookbook* , AN5413, Rev. 5, 2020.
- [3] Arduino, *Analog In, Out Serial* , Built-in Examples.
- [4] EduFramework, *Analog API* , EduFramework Source Code.